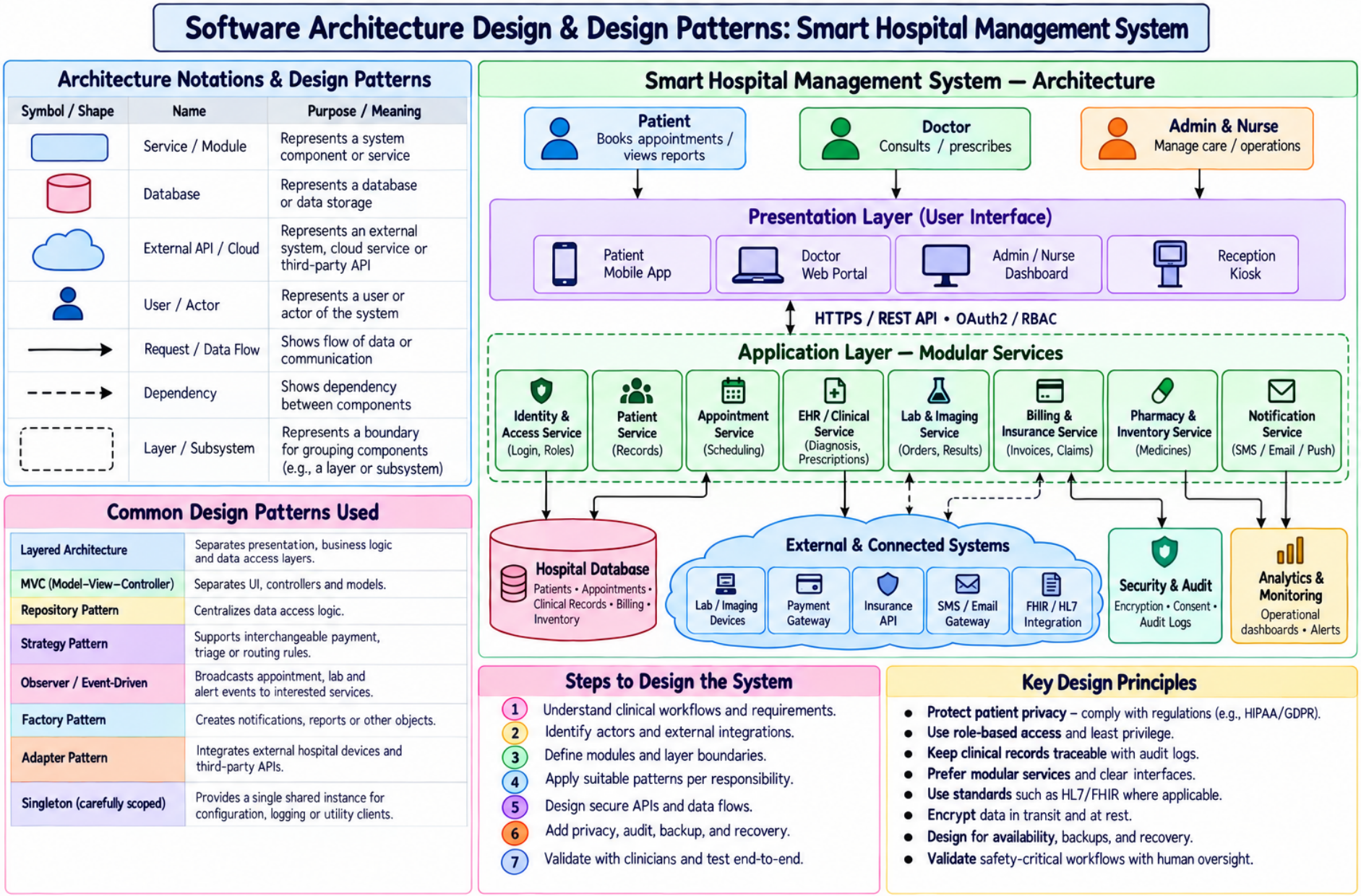
Figure 1: Smart Hospital Management System Architecture Diagram



Description of the Diagram

1. Overview

This document describes a Smart Hospital Management System using three diagrams. Figure 1 is the architecture diagram, with its legend, layers, services, design patterns, design steps and key principles. Figure 2 is the class diagram, which shows the classes, their data and operations, and how they relate. Figure 3 is the use case diagram, which shows which features each actor can use.

The system is split into three main parts: a Presentation Layer (what users see), an Application Layer (the services that do the work) and a Data and External Systems layer (storage and outside connections). Each part is explained below in short.

2. Architecture Notations (Legend)

These symbols explain how to read the architecture diagram (Figure 1).

Symbol / Shape	Meaning
Service / Module	A blue rounded box. Represents one system component or service that does a specific job.
Database	A red/pink cylinder. Represents a database or data storage.
External API / Cloud	A blue cloud. Represents an outside system, cloud service or third-party API.
User / Actor	A person icon. Represents a user or actor who uses the system.
Request / Data Flow	A solid arrow. Shows the flow of data or communication between parts.
Dependency	A dashed arrow. Shows that one component depends on another.
Layer / Subsystem	A dashed rounded box. Groups components into a boundary such as a layer or subsystem.

3. Users (Actors) and Their Features

The system has seven actors. They all inherit from a common User class, so each one can log in, log out, update their profile and change their password. The table below shows what each actor does, based on the use case diagram (Figure 3).

Actor	Role	Features (Use Cases)
Patient	Receives care and uses hospital services.	Book Appointment, View Appointment, Cancel Appointment, View Medical Records, Make Payment
Doctor	Examines patients and decides treatment.	Diagnose Patient, Generate Prescription, Request Lab Test
Nurse	Supports patient care on the ward.	Manage Patient Profile
Admin	Runs and controls the hospital system.	Manage Doctors, Manage Staff, Generate Reports
Receptionist	Front desk: registration and billing.	Register Patient, Generate Bill
Laboratory Staff	Runs tests and records results.	Conduct Lab Test, Update Lab Results
Pharmacist	Looks after medicines and stock.	Manage Medicines

Class details of each actor

Based on the class diagram (Figure 2): the data each class stores (attributes) and the actions it can perform (operations).

Class	Attributes (Data)	Operations (Functions)
User (base class)	userId, username, passwordHash, email, phone, role	login(), logout(), updateProfile(), changePassword()
Patient	patientId, name, dateOfBirth, gender, address, bloodGroup	registerPatient(), bookAppointment(), viewMedicalRecords(), cancelAppointment(), makePayment()
Doctor	doctorId, specialization, qualification, experienceYears, consultationFee	diagnosePatient(), generatePrescription(), requestLabTest()
Nurse	nurseId, qualification, shift, department	managePatientProfile(), updatePatientInformation()
Admin	adminId, accessLevel	manageDoctors(), manageStaff(), generateReports()
Receptionist	receptionistId, shift, deskNumber	registerPatient(), scheduleAppointment(), generateBill()
LaboratoryStaff	labStaffId, qualification, labDepartment	conductLabTest(), updateLabResults()
Pharmacist	pharmacistId, qualification, licenseNumber	manageMedicines(), updateMedicineStock(), dispenseMedicine()

Use case relationships

The dashed arrows marked include in the use case diagram mean one feature always uses another.

Use Case	Meaning
Login (shared)	Login is available to all actors.
Make Payment includes Generate Bill	A payment always needs a bill, so the bill generation step is included.
Diagnose Patient includes View Medical Records	A doctor reviews the patient's medical records while diagnosing.
Request Lab Test includes Conduct Lab Test	Requesting a test leads to the lab conducting it.

4. Presentation Layer (User Interface)

This layer is what users interact with. It contains four interfaces:

Interface	Purpose
Patient Mobile App	Phone app used by patients for appointments and reports.
Doctor Web Portal	Browser-based portal where doctors view records and prescribe.
Admin / Nurse Dashboard	Screen for staff to manage care, beds, schedules and operations.
Reception Kiosk	Self-service touch screen at the hospital for check-in and registration.

5. Communication and Security Terms

The arrow between the Presentation Layer and the Application Layer is labelled with these terms:

Term	Meaning
HTTPS	Secure web protocol that encrypts data travelling between the UI and the services.
REST API	Standard way for the UI to request data and actions from the services over HTTP.
OAuth2	Secure login and authorization standard; lets users access the system with tokens instead of sharing passwords.
RBAC	Role-Based Access Control: what a user can do depends on their role (doctor, nurse, admin, patient).

6. Application Layer: Modular Services

The Application Layer is a set of small, independent services. Each service has one clear responsibility.

Service	Function
Identity & Access Service	Handles login, user roles and permissions.
Patient Service	Manages patient records and basic patient details.
Appointment Service	Handles scheduling: booking, rescheduling and cancelling appointments.
EHR / Clinical Service	Electronic Health Record service. Stores diagnosis and prescriptions.
Lab & Imaging Service	Manages lab and imaging orders and their results.
Billing & Insurance Service	Creates invoices and processes insurance claims.
Pharmacy & Inventory Service	Tracks medicines and stock levels in the pharmacy.
Notification Service	Sends SMS, email and push alerts (reminders, results, updates).

7. Data, External Systems and Support Components

The bottom of the architecture shows where data is stored, which outside systems are connected, and which support components protect and monitor the system.

Component	Function
Hospital Database	Central storage for patients, appointments, clinical records, billing and inventory data.
Lab / Imaging Devices	External medical equipment (scanners, analysers) that sends results to the Lab & Imaging Service.
Payment Gateway	Third-party service that processes online payments for bills.
Insurance API	External interface to insurance companies for verifying coverage and submitting claims.
SMS / Email Gateway	External provider that actually delivers SMS and email messages.
FHIR / HL7 Integration	Healthcare data standards that let the system exchange records with other hospitals and systems.
Security & Audit	Encryption, consent management and audit logs that record who accessed what.

Component	Function
Analytics & Monitoring	Operational dashboards and alerts to watch system health and hospital performance.

8. Data Classes (Class Diagram)

Besides the actor classes, the class diagram (Figure 2) has eight classes that hold hospital data.

Class	Description
Appointment	A booked visit with a doctor. Can be booked, rescheduled, cancelled or confirmed. Holds date, time, status and reason.
MedicalRecord	A patient's health history: diagnosis, allergies, notes and record date. Can be created, updated and viewed.
Prescription	Medicines ordered by a doctor, with dosage, duration and instructions. Can be created, updated and viewed.
LabTest	A requested test with name, status, result and dates. Can be requested, conducted and updated with results.
Medicine	A pharmacy item with name, manufacturer, unit price, stock quantity and expiry date. Can be added, updated and stock-checked.
Bill	An invoice with date, total, tax and status. Can be generated, totalled and viewed.
Payment	A payment made against a bill: amount, method, transaction status and reference. Can be processed, verified and refunded.
Report	A hospital report with type, period and generated date. Can be generated, exported and viewed.

Relationships between classes

The labelled lines in the class diagram show who does what. Numbers such as 1 and 0..* mean one and zero-or-many.

Relationship	Meaning
User (generalization)	Nurse, Admin, Receptionist, Patient, Doctor, Laboratory Staff and Pharmacist all inherit from User, so every one of them can log in, log out, update a profile and change a password.

Relationship	Meaning
Admin generates Report	One admin generates many reports.
Receptionist generates Bill	The receptionist creates bills for patients.
Patient receives Bill	A patient can receive many bills.
Patient books Appointment	A patient can book many appointments.
Patient owns MedicalRecord	Each patient has their own medical records.
Patient receives Prescription	A patient can receive many prescriptions.
Patient undergoes LabTest	A patient can have many lab tests.
Doctor attends Appointment	A doctor attends many appointments.
Doctor creates MedicalRecord	A doctor writes the medical records.
Doctor issues Prescription	A doctor issues prescriptions.
Doctor requests LabTest	A doctor requests lab tests.
Laboratory Staff performs LabTest	Lab staff carry out the tests.
Bill contains Payment (composition)	A payment belongs to a bill and cannot exist without it.
Prescription prescribes Medicine	A prescription lists the medicines to be given.
Pharmacist manages Medicine	The pharmacist maintains medicines and stock.

9. Common Design Patterns Used

A design pattern is a proven, reusable solution to a common design problem.

Pattern	Purpose
Layered Architecture	Separates presentation, business logic and data access into layers.
MVC (Model-View-Controller)	Separates the user interface, the controllers and the data models.

Pattern	Purpose
Repository Pattern	Keeps all data access logic in one place, so services do not talk to the database directly.
Strategy Pattern	Allows swapping rules at runtime, such as different payment methods or triage/ routing rules.
Observer / Event-Driven	Broadcasts events (new appointment, lab result, alert) to the services that need them.
Factory Pattern	Creates objects such as notifications or reports without exposing how they are built.
Adapter Pattern	Connects external hospital devices and third-party APIs to the system through a common interface.
Singleton (carefully scoped)	Provides one shared instance for configuration, logging or utility clients; used sparingly.

10. Steps to Design the System

The architecture diagram lists seven steps to follow when designing this kind of system.

Step	What to do
1. Understand requirements	Understand clinical workflows and requirements.
2. Identify actors and integrations	Find who uses the system and which external systems it must connect to.
3. Define modules and layers	Decide the modules and the layer boundaries.
4. Apply patterns	Choose suitable design patterns for each responsibility.
5. Design secure APIs and flows	Plan secure APIs and data flows.
6. Add privacy, audit, backup, recovery	Build in privacy, audit trails, backup and disaster recovery.
7. Validate and test	Validate with clinicians and test the system end to end.

11. Key Design Principles

These principles guide every decision in the design.

Principle	Meaning
Protect patient privacy	Comply with regulations such as HIPAA and GDPR.
Use role-based access	Give each user only the access they need (least privilege).
Keep records traceable	Keep clinical records traceable with audit logs.
Prefer modular services	Use modular services with clear interfaces.
Use standards	Use standards such as HL7/FHIR where applicable.
Encrypt data	Encrypt data in transit and at rest.
Design for availability	Plan for availability, backups and recovery.
Human oversight	Validate safety-critical workflows with human oversight.

12. Summary

Seven kinds of users (patient, doctor, nurse, admin, receptionist, laboratory staff and pharmacist) reach the system through mobile, web, dashboard or kiosk interfaces. Each has its own features, such as booking appointments, diagnosing patients, managing medicines or running lab tests. Requests travel securely (HTTPS, REST, OAuth2, RBAC) to modular services that manage identity, patients, appointments, clinical records, labs, billing, pharmacy and notifications. These services use a central hospital database and connect to external devices, payment, insurance, messaging and FHIR/HL7 systems. Security and audit components protect data, while analytics and monitoring track performance. Well-known design patterns and clear principles keep the system secure, maintainable and safe for patients.